

A person in a dark suit and tie is shown from the chest up, interacting with a futuristic, glowing blue digital interface. The interface is centered around a circular hub with the text "DATA ANALYSIS". Surrounding this hub are various icons representing data and technology: a bar chart, a line graph, a globe, a pie chart, a gear, and a computer monitor. The person's right hand is reaching out, touching one of the gear icons. The background is dark, making the glowing interface elements stand out.

**DATA
ANALYSIS**

A person in a dark suit and tie is shown from the chest up, interacting with a futuristic, glowing blue digital interface. The interface is centered around a circular hub with the text "DATA ANALYSIS". Surrounding this hub are various icons representing data and technology: a bar chart, a line graph, a globe, a pie chart, a gear, and a computer monitor. The person's right hand is reaching out, touching one of the gear icons. The background is dark, making the glowing interface elements stand out.

**DATA
ANALYSIS**

OOP in Python

To map with real world scenarios, we started using objects in code.

This is called **object oriented programming**.

Class & Object in Python

Class is a blueprint for creating objects.

#creating class

```
class Student:  
    name = "karan kumar"
```

#creating object (instance)

```
s1 = Student()  
print( s1.name )
```

Class & Instance Attributes

Class.attr

obj.attr

Apna College

__init__ Function

Constructor

All classes have a function called `__init__()`, which is always executed when the object is being initiated.

#creating class

```
class Student:  
    def __init__( self, fullname ):  
        self.name = fullname
```

#creating object

```
s1 = Student( "karan" )  
print( s1.name )
```

*The **self** parameter is a reference to the current instance of the class, and is used to access variables that belongs to the class.

Methods

Methods are functions that belong to objects.

#creating class

```
class Student:
    def __init__( self, fullname ):
        self.name = fullname

    def hello( self ):
        print( "hello", self.name)
```

#creating object

```
s1 = Student( "karan" )
s1.hello( )
```

Let's Practice

Create student class that takes name & marks of 3 subjects as arguments in constructor.
Then create a method to print the average.

Apna College

Static Methods

Methods that don't use the self parameter (work at class level)

```
class Student:
```

```
    @staticmethod    #decorator
```

```
    def college( ):
```

```
        print( "ABC College" )
```

*Decorators allow us to wrap another function in order to extend the behaviour of the wrapped function, without permanently modifying it

Important

Abstraction

Hiding the implementation details of a class and only showing the essential features to the user.

Encapsulation

Wrapping data and functions into a single unit (object).

Let's Practice

Create Account class with 2 attributes - balance & account no.

Create methods for debit, credit & printing the balance.

Apna college

A person in a dark suit and tie is shown from the chest up, interacting with a futuristic, glowing blue digital interface. The interface is centered around a circular hub with the text "DATA ANALYSIS". Surrounding this hub are various icons representing data and technology: a bar chart, a line graph, a globe, a pie chart, a gear, and a computer monitor. The person's right hand is reaching out, touching one of the gear icons. The background is dark, making the glowing interface elements stand out.

**DATA
ANALYSIS**

A person in a dark suit and tie is shown from the chest up, interacting with a futuristic, glowing blue digital interface. The interface is centered around a circular hub with the text "DATA ANALYSIS". Surrounding this hub are various icons representing data and technology: a bar chart, a line graph, a globe, a pie chart, a gear, and a computer monitor. The person's right hand is reaching out, touching one of the gear icons. The background is dark, making the glowing interface elements stand out.

**DATA
ANALYSIS**

OOP in Python

To map with real world scenarios, we started using objects in code.

This is called **object oriented programming**.

Class & Object in Python

Class is a blueprint for creating objects.

#creating class

class Student:

 name = "karan kumar"

#creating object (instance)

s1 = Student()

print(s1.name)

Class & Instance Attributes

Class.attr

obj.attr

Apna College

__init__ Function

Constructor

All classes have a function called `_init_()`, which is always executed when the object is being initiated.

#creating class

```
class Student:  
    def __init__( self, fullname ):  
        self.name = fullname
```

#creating object

```
s1 = Student( "karan" )  
print( s1.name )
```

*The **self** parameter is a reference to the current instance of the class, and is used to access variables that belongs to the class.

Methods

Methods are functions that belong to objects.

#creating class

```
class Student:  
    def __init__( self, fullname ):  
        self.name = fullname  
  
    def hello( self ):  
        print( "hello", self.name)
```

#creating object

```
s1 = Student( "karan" )  
s1.hello( )
```

Let's Practice

Create student class that takes name & marks of 3 subjects as arguments in constructor.
Then create a method to print the average.

Apna College

Static Methods

Methods that don't use the self parameter (work at class level)

```
class Student:
```

```
    @staticmethod    #decorator
```

```
    def college( ):
```

```
        print( "ABC College" )
```

*Decorators allow us to wrap another function in order to extend the behaviour of the wrapped function, without permanently modifying it

Important

Abstraction

Hiding the implementation details of a class and only showing the essential features to the user.

Encapsulation

Wrapping data and functions into a single unit (object).

Let's Practice

Create Account class with 2 attributes - balance & account no.

Create methods for debit, credit & printing the balance.

Apna college